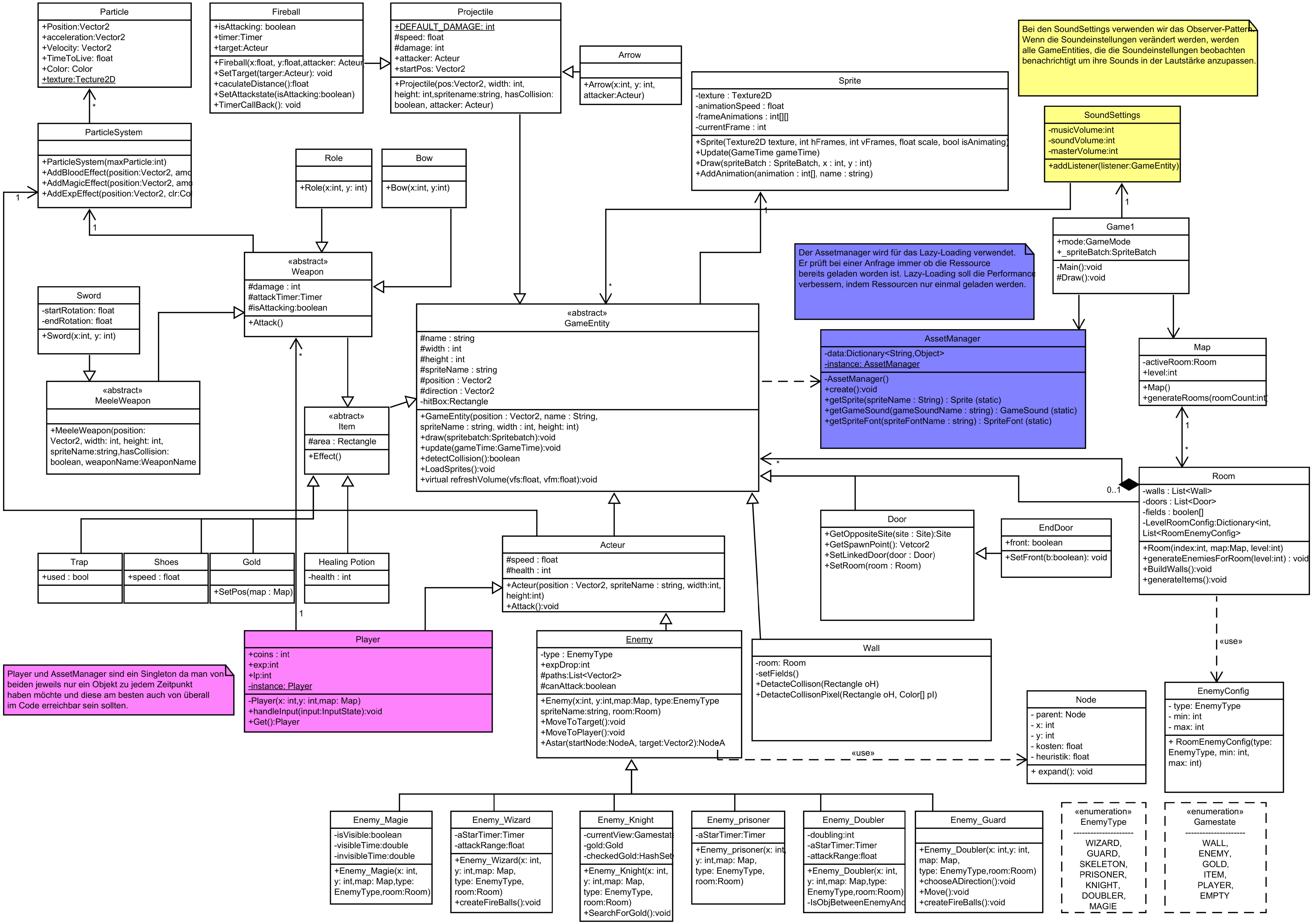
Particle
+Position:Vector2
+acceleration:Vector2
+Velocity: Vector2
+TimeToLive: float
+Color: Color
+texture:Tecture2D
Fireball
+isAttacking: boolean
+timer:Timer
+target:Acteur
+Fireball(x:float, y:float,attacker: Acteur
+SetTarget(targer:Acteur): void
+caculateDistance():float
+SetAttackstate(isAttacking:boolean)
+TimerCallBack(): void
Projectile
+DEFAULT_DAMAGE: int
#speed: float
#damage: int
+attacker: Acteur
+startPos: Vector2
+Projectile(pos:Vector2, width: int, height: int,spritename:string, hasCollision: boolean, attacker: Acteur)
Arrow
+Arrow(x:int, y: int, attacker:Acteur)
Sprite
-texture : Texture2D
-animationSpeed : float
-frameAnimations : int[][]
-currentFrame : int
+Sprite(Texture2D texture, int hFrames, int vFrames, float scale, bool isAnimating)
+Update(GameTime gameTime)
+Draw(spriteBatch : SpriteBatch, x : int, y : int)
+AddAnimation(animation : int[], name : string)
Bei den SoundSettings verwenden wir das Observer-Pattern. Wenn die Soundeinstellungen verändert werden, werden alle GameEntities, die die Soundeinstellungen beobachten benachrichtigt um ihre Sounds in der Lautstärke anzupassen.
SoundSettings
-musicVolume:int
-soundVolume:int
-masterVolume:int
+addListener(listener:GameEntity)
ParticleSystem
+ParticleSystem(maxParticle:int)
+AddBloodEffect(position:Vector2, amo
+AddMagicEffect(position:Vector2, amo
+AddExpEffect(position:Vector2, clr:Co
Role
+Role(x:int, y: int)
Bow
+Bow(x:int, y:int)
Game1
+mode:GameMode
+_spriteBatch:SpriteBatch
-Main():void
#Draw():void
«abstract»
Weapon
#damage : int
#attackTimer:Timer
#isAttacking:boolean
+Attack()
Der Assetmanager wird für das Lazy-Loading verwendet. Er prüft bei einer Anfrage immer ob die Ressource bereits geladen worden ist. Lazy-Loading soll die Performance verbessern, indem Ressourcen nur einmal geladen werden.
Sword
-startRotation: float
-endRotation: float
+Sword(x:int, y: int)
«abstract»
GameEntity
#name : string
#width : int
#height : int
#spriteName : string
#position : Vector2
#direction : Vector2
-hitBox:Rectangle
+GameEntity(position : Vector2, name : String, spriteName : string, width : int, height: int)
+draw(spritebatch:Spritebatch):void
+update(gameTime:GameTime):void
+detectCollision():boolean
+LoadSprites():void
+virtual refreshVolume(vfs:float, vfm:float):void
AssetManager
-data:Dictionary<String,Object>
-instance: AssetManager
-AssetManager()
+create():void
+getSprite(spriteName : String) : Sprite (static)
+getGameSound(gameSoundName : string) : GameSound (static)
+getSpriteFont(spriteFontName : string) : SpriteFont (static)
Map
-activeRoom:Room
+level:int
+Map()
+generateRooms(roomCount:int
«abstract»
MeeleWeapon
+MeeleWeapon(position: Vector2, width: int, height: int, spriteName:string,hasCollision: boolean, weaponName:WeaponName
«abtract»
Item
#area : Rectangle
+Effect()
Room
-walls : List<Wall>
-doors : List<Door>
-fields : boolen[]
-LevelRoomConfig:Dictionary<int, List<RoomEnemyConfig>
+Room(index:int, map:Map, level:int)
+generateEnemiesForRoom(level:int) : void
+BuildWalls():void
+generateItems():void
Door
+GetOppositeSite(site : Site):Site
+GetSpawnPoint(): Vetcor2
+SetLinkedDoor(door : Door)
+SetRoom(room : Room)
EndDoor
+front: boolean
+SetFront(b:boolean): void
Trap
+used : bool
Shoes
+speed : float
Gold
+SetPos(map : Map)
Healing Potion
-health : int
Acteur
#speed : float
#health : int
+Acteur(position : Vector2, spriteName : string, width:int, height:int)
+Attack():void
Player
+coins : int
+exp:int
+lp:int
-instance: Player
-Player(x: int,y: int,map: Map)
+handleInput(input:InputState):void
+Get():Player
Player und AssetManager sind ein Singleton da man von beiden jeweils nur ein Objekt zu jedem Zeitpunkt haben möchte und diese am besten auch von überall im Code erreichbar sein sollten.
Enemy
-type : EnemyType
+expDrop:int
#paths:List<Vector2>
#canAttack:boolean
+Enemy(x:int, y:int,map:Map, type:EnemyType spriteName:string, room:Room)
+MoveToTarget():void
+MoveToPlayer():void
+Astar(startNode:NodeA, target:Vector2):NodeA
Wall
-room: Room
-setFields()
+DetacteCollison(Rectangle oH)
+DetacteCollisonPixel(Rectangle oH, Color[] pl)
«use»
Node
- parent: Node
- x: int
- y: int
- kosten: float
- heuristik: float
+ expand(): void
EnemyConfig
- type: EnemyType
- min: int
- max: int
+ RoomEnemyConfig(type: EnemyType, min: int, max: int)
Enemy_Magie
-isVisible:boolean
-visibleTime:double
-invisibleTime:double
+Enemy_Magie(x: int, y: int,map: Map,type: EnemyType,room:Room)
Enemy_Wizard
-aStarTimer:Timer
-attackRange:float
+Enemy_Wizard(x: int, y: int,map: Map, type: EnemyType, room:Room)
+createFireBalls():void
Enemy_Knight
-currentView:Gamestat
-gold:Gold
-checkedGold:HashSet
+Enemy_Knight(x: int, y: int,map: Map, type: EnemyType, room:Room)
+SearchForGold():void
Enemy_prisoner
-aStarTimer:Timer
+Enemy_prisoner(x: int, y: int,map: Map, type: EnemyType, room:Room)
Enemy_Doubler
-doubling:int
-aStarTimer:Timer
-attackRange:float
+Enemy_Doubler(x: int, y: int,map: Map,type: EnemyType,room:Room)
-IsObjBetweenEnemyAnd
Enemy_Guard
+Enemy_Doubler(x: int,y: int, map: Map, type: EnemyType,room:Room)
+chooseADirection():void
+Move():void
+createFireBalls():void
«enumeration»
EnemyType
WIZARD,
GUARD,
SKELETON,
PRISONER,
KNIGHT,
DOUBLER,
MAGIE
«enumeration»
Gamestate
WALL,
ENEMY,
GOLD,
ITEM,
PLAYER,
EMPTY